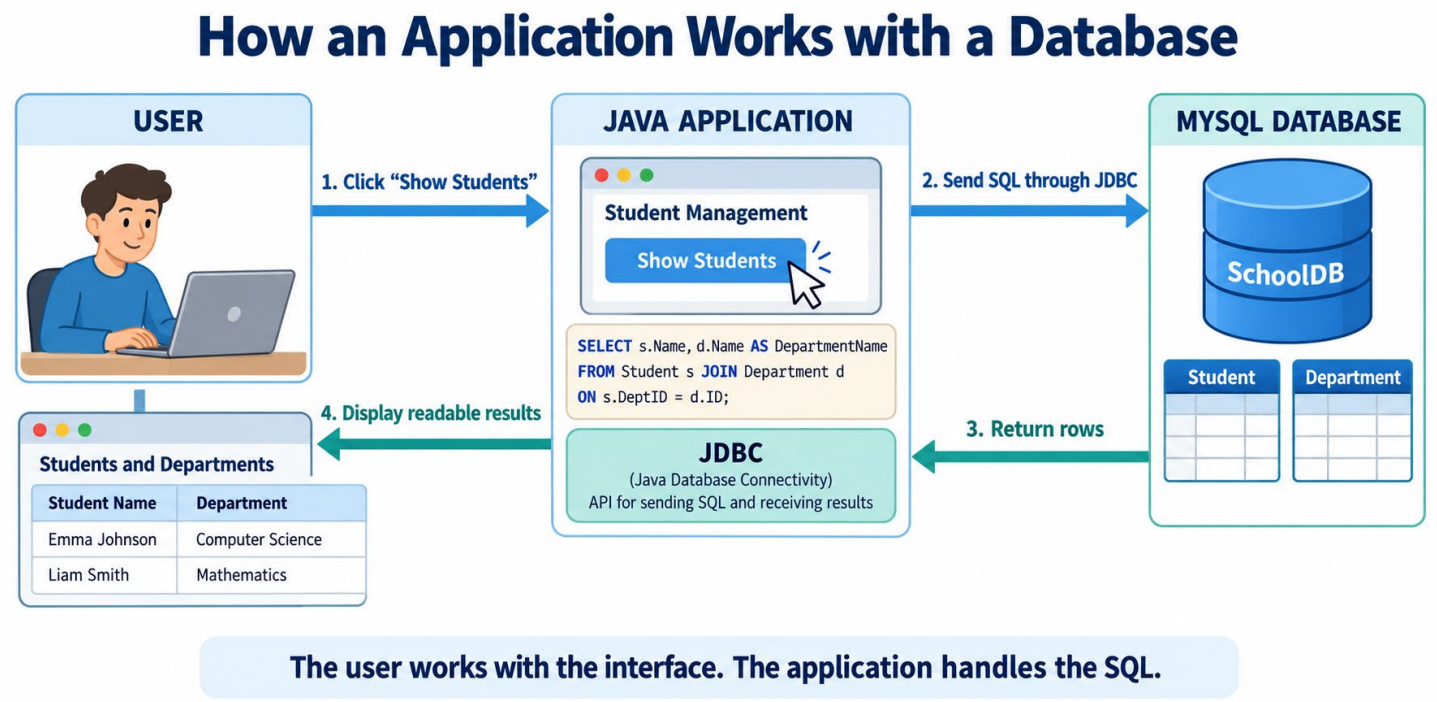
JDBC overview

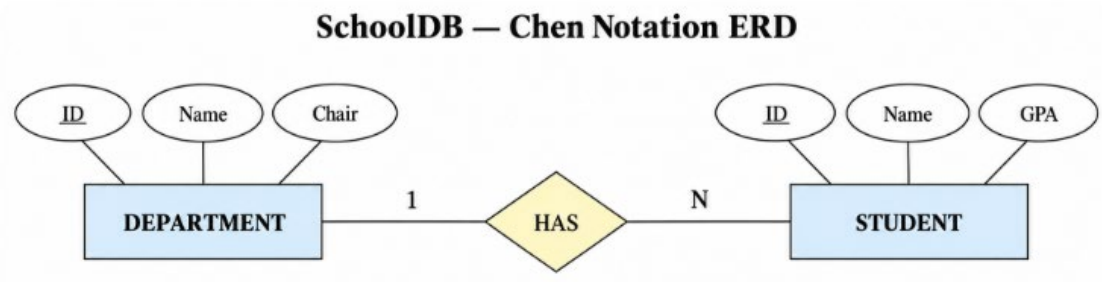
JDBC stands for **Java Database Connectivity**. It is Java's standard API for communicating with databases. A Java program uses JDBC to connect to a database, send SQL statements, and process the results.

A **typical database user does not write SQL statements directly**. Instead, typically, the database and tables is created by the database administrator and the user works through an interface provided by an application, such as a **form, menu, or search screen**. The application translates the user's actions into SQL statements, sends them to the database, and displays the results in a readable format.

In a Java application, **JDBC provides the connection between the program and the database**. For example, in the Course Enrollment program, a user can enter a student's information into a form and click **Insert**. The program collects the values, uses JDBC to execute an SQL `INSERT` statement, and displays a confirmation. **The user does not need to know the SQL syntax or the structure of the underlying tables.**



The `SchoolDB` sample demonstrates this process through a console program. It connects to the `SchoolDB` database, retrieves students and their departments, and prints the results in columns.



JDBC components used in the program

The program imports five JDBC types from the `java.sql` package:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

Each has a specific responsibility:

JDBC type	Purpose in this program
`DriverManager`	Opens the database connection.
`Connection`	Represents the connection to MySQL.
`PreparedStatement`	Executes the SQL query.
`ResultSet`	Holds the query results for the program to read.
`SQLException`	Represents errors that occur during database operations.

Establishing a database connection

The program first defines the connection information:

```
String url = "jdbc:mysql://localhost:3306/SchoolDB";
String username = "root";
String password = "root";
```

The URL identifies how and where to connect:

- `jdbc:mysql:` specifies a JDBC connection to MySQL.
- `localhost` identifies a database server running on the same computer.
- `3306` is the server's port number.
- `SchoolDB` is the database to use.

The following statement opens the connection using those settings:

```
Connection connection =
        DriverManager.getConnection(url, username, password);
```

If successful, it returns a `Connection` object. The MySQL Connector/J driver must be available to the application for this connection to work.

Defining the SQL query

The program stores its query in a Java string:

```
String sql =
        "SELECT s.ID, s.Name, d.Name AS DepartmentName " +
        "FROM Student s " +
        "JOIN Department d " +
        "ON s.DeptID = d.ID ";
```

This query retrieves each matching student's ID, name, and department name. The aliases `s` and `d` provide shorter names for the `Student` and `Department` tables.

The join matches the student's `DeptID` with the department's `ID`. Because this is an inner join, students without a matching department are excluded. For example, Lucas Harris from the sample data would not appear because his `DeptID` is `NULL`.

The alias `DepartmentName` gives the department name a distinct result column label, allowing the Java program to distinguish it from the student's `Name`.

Preparing and executing the query

The program creates a `PreparedStatement` using the SQL string:

```
PreparedStatement statement =
        connection.prepareStatement(sql);
```

It then executes the query:

```
ResultSet result = statement.executeQuery();
```

`executeQuery()` is used for SQL statements that return rows, such as `SELECT`. It returns a `ResultSet` containing the retrieved data.

This particular query has no parameters. If the program later allowed a user to search for a specific student, a `PreparedStatement` could also support `?` placeholders and methods such as `setInt()` to supply the search value.

Reading the results

The program reads the returned rows using a loop:

```
while (result.next()) {
    int studentID = result.getInt("ID");
    String studentName = result.getString("Name");
    String departmentName =
            result.getString("DepartmentName");

    System.out.printf(
            "%-10d %-25s %-25s%n",
            studentID,
            studentName,
            departmentName
    );
}
```

A `ResultSet` cursor initially sits before the first row. Each call to `next()` advances to the next row. It returns `true` when a row is available and `false` when there are no more rows.

The getter methods retrieve values from the current row:

- `getInt("ID")` retrieves the student ID as an integer.
- `getString("Name")` retrieves the student's name.
- `getString("DepartmentName")` retrieves the department's name.

The program then uses `System.out.printf()` to display these values in aligned columns. This formatting is ordinary Java output code; JDBC supplies the data being displayed.

Closing database resources

The connection, statement, and result set are declared inside a **try-with-resources** block:

```
try (
        Connection connection =
                DriverManager.getConnection(
                        url, username, password);

        PreparedStatement statement =
                connection.prepareStatement(sql);

        ResultSet result =
                statement.executeQuery()
) {
    // Display the results.
}
```

Java automatically closes these resources when the block finishes, including when an exception occurs. They close in reverse declaration order: the result set, the statement, and then the connection.

Handling database errors

The program catches database errors with:

```
catch (SQLException e) {
    System.out.println("Database error:");
    System.out.println(e.getMessage());
}
```

An `SQLException` can occur if the server is unavailable, the credentials are incorrect, the database or tables do not exist, or the SQL statement fails. The catch block displays the database error message so the user can see why the operation could not be completed.